

CHAPTER: 4

★ "LINKED LISTS" ★

LINKED LISTS:-

Definition:-

"linked list is a data structure made of nodes, nodes are non-contiguous unlike an array. Each node contains a data and a pointer to the next node."

Explanation:-

Remember how an array or a vector requires contiguous memory locations? Well lists don't. A list or linked list contains two classes, Node and list. You can also call this Node as ListNode for this chapter.

NODE :-

Definition:-

"A Node is an object which contains two things, a data and a pointer."

Date: / /

Day

Data :-

This data can be an integer, character, float etc. It is whatever datatype is needed.

Pointer :-

The pointer in a Node is used to point to another Node. In the context of a linked list, such a pointer is called next pointer. It shows the memory address of next node.

LINKED PROPERTY :-

A list / linked list is formed by chaining multiple ListNodes together in such a way that the next pointer of first Node holds the address of second Node, the next pointer of second Node hold the address of third Node and so on. The next pointer of last Node points to "NULL" showing the end of list.

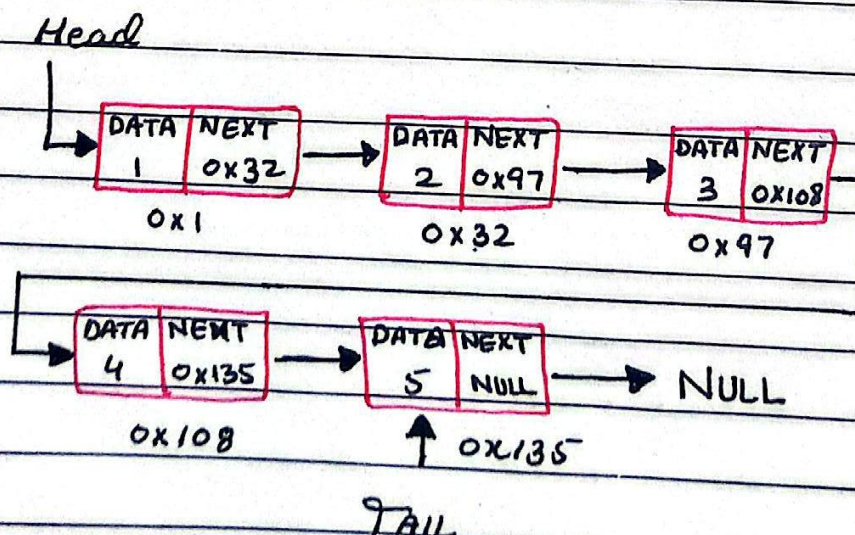
A list class is used to connect (insert / delete) ListNode's objects in a linked list.

LIST :-

List class contains usually one or in some cases two pointers. These are head and tail pointer. A head pointer is used to hold the address of first ListNode and the tail pointer is used to hold the address of last ListNode.

Every list must have a head pointer but tail pointer may or may not be present. If head is present, we can find the tail by constantly travelling to next pointers until we reach a next pointer which is pointing to "NULL". At this point we are on last ListNode which is also a tail.

MEMORY DIAGRAM:-



TYPES OF LINKED LIST:-

There are three types of linked list:-

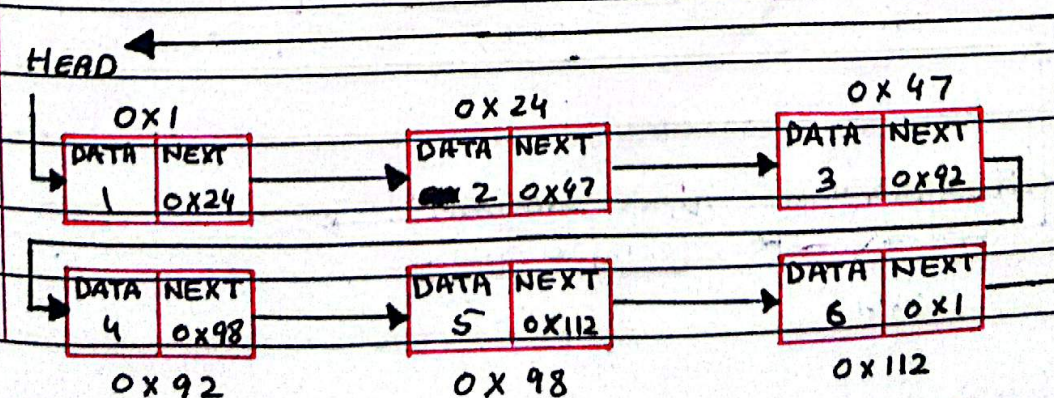
- 1) Singly Linked List
- 2) Circlly Linked List
- 3) Doubly Linked List

Up until now, we were talking about singly linked list but the difference is not major:-

CIRCLY LINKED LIST:-

Circlly linked list has only one difference. The next pointer of its last Node does not point to "NULL", instead, it points back to "Head". Hence, the list forms a ring shape and this why it is called a circlly linked list.

MEMORY DIAGRAM:-

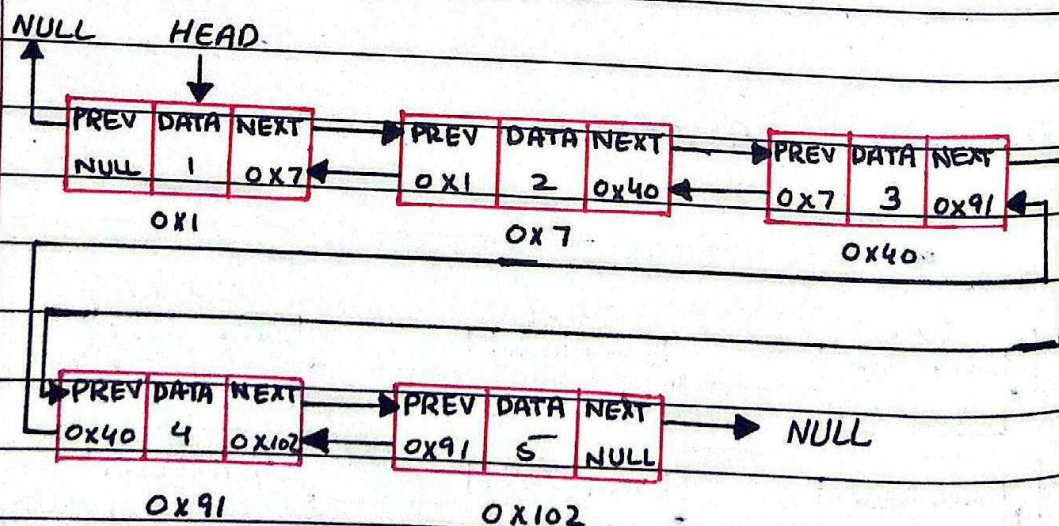


DOUBLY LINKED LIST:-

Doubly linked list has an additional pointer called previous, in its Node class. Just as next pointer holds the address of next Node, the previous pointer holds the address of previous Node.

This property gives birth to a power in doubly linked list. In a doubly linked list, you can traverse both forward (using next) and backward (using previous). In a singly linked list or cyclic linked list, you can only move forwards. Hence, you have more mobility in a doubly linked list.

MEMORY DIAGRAM:-



Date: ___/___/___

Day _____

This marks the end of definitions and now we will dive into the code of `ListNode` and `list` class to understand the internal working of a linked list. For now, we will use the singly linked list as our test subject. Let's look at `ListNode` first.

Code :-

```
class ListNode {  
public:  
    int data;  
    ListNode* next;  
    ListNode(int data): data(data), next(NULL) {}  
    // you can also use "nullptr" as "NULL".  
};
```

The `int data` holds the value passed to constructor at the time of Node declaration. Constructor gives this value to the `data` variable. Constructor also sets the `next` pointer to "NULL" by default until we point it to another `ListNode`. Now using what you have learnt, try making a `listNode` class for a doubly linked list.

Date: __/__/__

Day __

That is all for the `ListNode` class, now let's look at the basic code of the list class and again we are going to be using the example of singly linked list -

Code -

```
class list {  
    ListNode* head; // Points to first ListNode  
    ListNode* tail; // Points to last ListNode  
public:  
    list(): head(NULL), tail(NULL) {}  
    ~list()  
    {  
        while(head)  
            pop-front(); // deletes first ListNode  
    }  
    // push-back, pop-back, push-front etc  
};
```

The `ListNode* head` points to first Node and every list must have a head pointer.

The `ListNode* tail` points to last Node but it is not necessary to have a ~~the~~ tail pointer as explained before. However, having a

Date: __/__/__

Day _____

tail pointer allows you direct access to list's last Node and this allows for extremely fast insertion or deletion at the back of linked list.

The constructor of a list does a single task, it sets the head and tail (if exists) to "NULL".

The destructor of a list deletes all the listNodes by repeatedly calling the pop-front method which deletes the Node which head points to, basically first Node of list. This keeps on going until the head becomes "NULL", which means all Nodes are now deleted.

Now that we have walked through the data members and the constructor / destructor of a list it's time to go through the major methods of a list (singly).

Following are the methods we will learn right now :-

- | | |
|------------------|------------------|
| 1) push-front(); | 2) pop-front(); |
| 3) push-back(); | 4) pop-back(); |
| 5) insertNode(); | 6) deleteNode(); |
| 7) searchNode(); | |

PUSH_FRONT:-

The push-front method is used to push a value from front of a linked list by creating a `ListNode` and setting its data to the passed value. Then pointing the next to head and the head to this new Node. This new Node becomes the first Node.

Code :-

```
void push-front(int data){  
    ListNode* newNode = new ListNode(data);  
    if (!head)  
    {  
        head = tail = newNode;  
    }  
    else  
    {  
        newNode->next = head;  
        head = newNode;  
    }  
}
```

It also checks if list is empty by seeing if head is "NULL", in this case it sets both head and tail to newNode.

POP-FRONT :-

The pop-front method is used to delete the current first Node and update head to second Node, which then becomes the first Node. This function is used in list's destructor to delete the chain of Nodes by repeatedly deleting head Node until head becomes "NULL". Its code is given :-

Code :-

```
void pop-front(){  
    if(!head) return;  
    ListNode* temp = head;  
    head = head->next;  
    delete temp;  
}
```

It first checks if list is empty by checking if head is "NULL", if so then it returns without doing anything. Otherwise it creates a temporary ListNode pointer to head, then it updates the head to second Node, and deletes temp. The second Node becomes the new first Node.

PUSH_BACK:-

The push-back method is used to insert a new Node at the back of list. This is the point where having a tail pointer helps the most. Since we need to insert a Node at the back, we can directly access the back using a tail pointer. Otherwise, we would have to traverse the entire list from head to last Node. Here is the code for pushback:-

Code :-

```
void push-back(int data){
    ListNode* newNode = new ListNode(data);
    if(!head)
    {
        head = tail = newNode;
    }
    else
    {
        tail->next = newNode;
        tail = newNode;
    }
}
```


POP-BACK:-

The pop-back method is used to delete the last Node by accessing the second last Node. First we set tail to second last Node then we delete tail's next and finally set tail's next to "NULL". Now since we need to access second last Node and since we can't go to previous Node in a singly linked list, our tail pointer won't help in speed here, we need to traverse the entire list from head to second last Node (which we need). Following is the code for pop-back method:-

Code:-

```
void popback(){
    if (!head) return;
    if (head == tail){ delete head;
        head = tail = NULL; return; }
    ListNode* temp = head;
    while (temp->next != tail) temp = temp->next;
    tail = temp;
    delete tail->next; tail->next = NULL;
}
```


INSERT NODE :-

The `insertNode` method is used to insert a new Node at a particular position in the list. It takes two arguments, a data and a position on which Node is to be inserted. It walks through list to reach position - 1 (Node before the target position) and inserts the new Node there:-

Code :-

```
void insertNode(int data, int pos){
    if(pos < 1) return;
    if(pos == 1 || !head){ pushFront(data); return; }
    int size = 0;
    ListNode* temp = head;
    while(temp){ size++; temp = temp->next; }
    if(pos > size + 1) return;
    if(pos == size + 1){ pushBack(data); return; }
    ListNode* newNode = new ListNode(data);
    temp = head;
    for(int i = 1; i < pos - 1; i++){
        temp = temp->next;
        newNode->next = temp->next;
        temp->next = newNode;
    }
}
```

It also checks that position is valid.

DELETE NODE:-

The `deleteNode` method is used to delete N^{th} `ListNode` in a linked list. It works by walking through the list to reach $(N-1)^{\text{th}}$ Node. Then set a target pointer on N^{th} Node using $(N-1)^{\text{th}}$ Node's next pointer. Then removes target from list by setting $(N-1)^{\text{th}}$ Node's next to target's next. And finally deletes target (N^{th} Node).

Code:-

```
void deleteNode(int pos){
    if(pos <= 1) return;
    if(pos == 1 || !head){ pop-front(); return; }
    int size = 0;
    ListNode* temp = head;
    while(temp){ size++; temp = temp->next; }
    if(pos > size) return;
    if(pos == size){ pop-back(); return; }
    temp = head;
    for(int i = 1; i < pos - 1; i++)
        temp = temp->next;
    ListNode* target = temp->next;
    temp->next = target->next;
    delete target;
}
```


SEARCHNODE:-

The searchNode method is used to search for a Node with a particular value. It takes value in argument. It walks the list from head towards tail, if it finds a match of data in a Node, it returns the position of that Node. If whole list is traversed, and no match is found, then 0 is returned to show that value doesn't exist and so does the Node.

Code :-

```
int searchNode(int data){  
    ListNode* Temp = head;  
    int pos = 1;  
    while(Temp){  
        if(Temp->data == data) return pos;  
        pos++;  
        Temp = Temp->next;  
    }  
    return 0;  
}
```

This was all for major methods of list.

Date: ___/___/___

Day ___

Now let's look at some basic methods of list -

FRONT:-

The front method returns the head's data.

Code -

```
int front() const {  
    if (!head) throw std::runtime_error("Empty");  
    return head->data;  
}
```

BACK:-

The back method returns the tail's data.

Code -

```
int back() const {  
    if (!head) throw std::runtime_error("Empty");  
    return tail->data;  
}
```

Front and back also exist in vectors but they check bounds instead of head.

EMPTY:-

Just like vector, list also has a method to check if list is empty by checking if head is "NULL".

Code:-

```
bool empty() const { return head == NULL; }
```

CLEAR:-

The clear method is basically like the list destructor but can be called on desire and even when list's scope has not ended.

Code:-

```
void clear() {  
    while (head)  
        pop-front();  
}
```

This was all for linked lists. Now for practice, you can try making all the codes we just learnt and other than those here are some extra practice problems.

PRACTICE PROBLEMS:-

- 1) Write a pop-back for a doubly linked list in which you don't traverse the whole list to reach the second last ListNode.
- 2) Create a list class in which list size (number of current Nodes) is tracked in real time and your insertNode and deleteNode don't need loops to calculate size each time.
- 3) Write a function called "ReverseList", which reverses the linked list by taking head in argument and also returns the new head after reversing.
- 4) Write push-front, pop-front, push-back, insertNode and deleteNode for a doubly linked list.
- 5) Write a function which prints all values of a cyclic linked list.

This is gonna be all for linked lists. As always you can find all given codes at:
"github.com/coder-Retio/Fast-University"

Go into DSA ➡ Linked List. Good Luck!